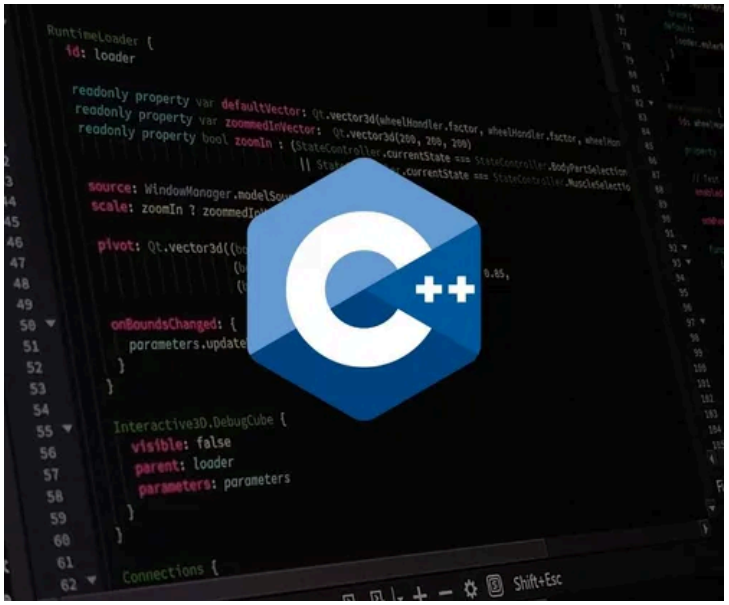
Materia: Laboratorio de Programación
EEST N.º 1 - “Eduardo Ader” Vicente López
5° año 3° Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs



**1° CUATRIMESTRE**
**📺🛠️🖋️ ACTIVIDAD 6: TALLER PRÁCTICO DE REPASO Y CONSOLIDACIÓN EN C++ - LPR 5° 3° A-B (Presencial/Virtual Asincrónico - Actividades de Contingencia y Continuidad Pedagógica) 2026**

## 🎯 OBJETIVOS DE LA ACTIVIDAD Y ALCANCE

Consolidar de forma práctica, analítica y sin fricción teórica los tres pilares fundamentales de bajo nivel trabajados durante la primera mitad del ciclo lectivo:

1. **Recursividad:** Comprensión de la Pila de Llamadas (***Stack***), flujo de llamadas autorreferenciales y la condición de corte obligatoria (*Caso Base*) para prevenir el colapso de memoria (***Stack Overflow***).
2. **Búsqueda Secuencial:** Recorrido indexado celda por celda en arreglos de memoria contigua RAM, optimizando el tiempo de ejecución de complejidad O(n) mediante interrupciones de ciclo (**break**).
3. **Punteros e Intercambio Físico:** Modificación directa de variables originales alojadas en la memoria mediante pasaje por dirección **(&)** y desreferenciación **(*)**.

### 💡 Conexión con el Proyecto Integrador Anual (Expotécnica 2026):

El control eficiente de la memoria de bajo nivel y el registro riguroso de versiones del software integrador debe asentarse en el archivo docs/**CHANGELOG.md** de sus repositorios grupales. Esta actividad provee las bases de software necesarias para procesar datos de sensores, módulos de hardware, etc. en tiempo real.

## 🧠 METODOLOGÍA DE APRENDIZAJE 5R

Para resolver esto aplicaremos el **Método 5R**:

1. **R1 - REGISTRAR (Copiar y Ejecutar):** Compilen y ejecuten el código base provisto en sus terminales de PowerShell dentro de VSCODE para observar la salida real en pantalla y las direcciones hexadecimales reales de la memoria **RAM**.
2. **R2 - REDUCIR (Identificar el Núcleo):** Identifiquen el "20% del código que genera el 80% del resultado" (el **if** del caso base en recursividad, el **break** de la búsqueda secuencial y la **variable** temporal del intercambio de punteros).
3. **R3 - RESPONDER (Demostrar Razonamiento):** Contesten las 5 preguntas del cuestionario técnico de control explicando con sus palabras ***por qué*** el programa actúa de esa manera a bajo nivel.
4. **R4 - REFLEXIONAR (Conectar con su Proyecto Integrador Anual):** Analicen si es viable cómo pueden aplicar esta misma lógica para por ejemplo leer datos de sensores, procesar buffers, etc. en el prototipo de su grupo si es que aplica para la Expotécnica.
5. **R5 - REVISAR (Verificar y Documentar):** Confirmen que no existan **errores de compilación**, realicen el **git push** a GitHub y actualicen la bitácora de la **carpeta de campo del proyecto integrador** en el archivo docs/CHANGELOG.md.

## 📕 GLOSARIO TÉCNICO DE BOLSILLO

Como futuros Técnicos, el vocabulario técnico preciso es su herramienta de presentación:

- **Recursividad: Función** que se invoca a sí misma para **resolver un problema dividiéndolo en instancias más pequeñas** del mismo tipo.
- **Caso Base:** Condición lógica escrita en una función recursiva para detener la cadena de llamadas y prevenir el desbordamiento de pila **(*Stack Overflow*)**.
- **Stack (Pila de Memoria):** Sector de la RAM donde el sistema operativo reserva espacio para las variables locales y el registro de retorno de cada función activa.
- **Búsqueda Secuencial:** Algoritmo de complejidad O(n) que recorre **índice por índice** un arreglo para comparar si el elemento buscado coincide con la celda actual.
- **Arreglo Contiguo:** Bloque de memoria RAM donde los elementos se almacenan en posiciones consecutivas, diferenciados por 4 bytes en variables enteras (**int**).
- **Puntero (type*):** Variable de bajo nivel que almacena la dirección de memoria física **hexadecimal** de otra variable.
- **Pasaje por Dirección (&):** Técnica mediante la cual pasamos la ubicación física (**&variable**) a un puntero para permitirle modificar la variable original fuera de su ámbito local.
- **Desreferenciación (*):** Operación para leer o escribir directamente sobre el **valor guardado** en la dirección de memoria apuntada por un puntero.

## 🗂️ ESTRUCTURA OBLIGATORIA DEL REPOSITORIO EN GITHUB

Para mantener el estándar de la actividad y facilitar el control de versiones con Git, la carpeta raíz llamada **repasogeneral** debe organizarse estrictamente de la siguiente manera:

```
repasogeneral/
├── .gitignore                  <-- Reglas de exclusión (.exe, .vscode/, *.out)
├── LICENSE                     <-- Licencia MIT de uso escolar
├── README.md                   <-- Portada explicativa con integrantes y menú de retos
├── docs/                       <-- Carpeta para entregables formales
│   └── InformeEEST1_LPR2026_ACT06_G99_Informe_v1.0.0.pdf <-- ENTREGABLE EN PDF (Normas APA v7)
│   └── manuales
│       ├── manual_programador_v1.0.0.pdf    <-- Basado en el ejemplo oficial
│       ├── manual_usuario_v1.0.0.pdf  <-- Basado en el ejemplo oficial
│   └── CHANGELOG.md              <-- Registro de cambios y versión SemVer/DocVer
├── src/                        <-- Código fuente modular en C++
│   └── main.cpp                <-- Suite unificada con los retos del taller
└── capturas/                   <-- Evidencias de ejecución local (.png)
    ├── ejecucion_repasogeneral.png       <-- Captura de la terminal ejecutando los retos
    └── traza_memoria.png         <-- Diagrama de bloques de la memoria RAM hecho a mano
```

## 💻 CÓDIGO FUENTE EN C++ (src/main.cpp)

```
/*
  ================================================================
  Materia: Laboratorio de Programación (LPR) — 5°
  Institución: E.E.S.T. N° 20 "Bangtan Sonyeondan" - Vicente López
  Archivo: src/main.cpp
  Actividad 6: Suite de Taller de Repaso e Integración
  ================================================================
*/

#include <iostream>

using namespace std;

// Prototipos de funciones modulares
int sumaRecursiva(int n);
void intercambiarValores(int* ptrA, int* ptrB);

int main() {
    cout << "=====================================================" << endl;
    cout << "  TALLER INTEGRADOR REPASO - ESTUDIANTE: Jung Kook " << endl;
    cout << "=====================================================" << endl;

    // --- RETO 1: RECURSIVIDAD ---
    cout << "\n=== RETO 1: SUMA RECURSIVA ===" << endl;
    int num = 5;
    cout << "Suma acumulada de 1 hasta " << num << ": " << sumaRecursiva(num) << endl;

    // --- RETO 2: ARRAYS Y BÚSQUEDA SECUENCIAL ---
```

```
    cout << "\n=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===" << endl;
    const int TAM = 10;
    int vectorDatos[TAM] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
    int numeroBuscar = 18;
    bool hallado = false;

    for (int i = 0; i < TAM; i++) {
        if (vectorDatos[i] == numeroBuscar) {
            cout << "[SISTEMA] Valor " << numeroBuscar << " hallado en el indice [" << i << "]" << endl;
            cout << "          Direccion RAM Hexadecimal: " << &vectorDatos[i] << endl;
            hallado = true;
            break; // Interrupción optimizada del ciclo O(n)
        }
    }

    if (!hallado) {
        cout << "[SISTEMA] El valor " << numeroBuscar << " no existe en el arreglo." << endl;
    }

    // --- RETO 3: INTERCAMBIO CON PUNTEROS ---
    cout << "\n=== RETO 3: INTERCAMBIO DE CELDAS ===" << endl;
    int x = 100, y = 500;
    cout << "Valores previos  -> X: " << x << " | Y: " << y << endl;

    // Invocación pasando direcciones de memoria
    intercambiarValores(&x, &y);

    cout << "Valores actuales -> X: " << x << " | Y: " << y << endl;
    cout << "====================================================" << endl;

    return 0; // Código 0: Ejecución exitosa
}

// Desarrollo de la función recursiva con caso base garantizado
int sumaRecursiva(int n) {
    if (n <= 0) return 0; // Caso base obligatorio para prevenir Stack Overflow
    return n + sumaRecursiva(n - 1); // Llamada recursiva
}

// Desarrollo del intercambio físico mediante desreferenciación
void intercambiarValores(int* ptrA, int* ptrB) {
    int temporal = *ptrA;
    *ptrA = *ptrB;
    *ptrB = temporal;
}
```

## 🏃 COMANDOS DE POWERSHELL EN VSCODE PARA COMPILAR Y EJECUTAR

# 1. Compilar el programa
```
g++ src/main.cpp -o src/repasogeneral.exe
```

# 2. Ejecutar la aplicación en Windows
```
.\src\repasogeneral.exe
```

## ⚙️🌐 ESTRATEGIA ANTI-CAÍDAS CÓMO PROTEGER NUESTRO PROYECTO: GITHUB + GITLAB (DUAL-REMOTE)

Imaginen que están a días de entregar su Proyecto Integrador Anual en la Expotécnica, y GitHub se cae o no pueden entrar a su cuenta. Perder el código de forma temporal o permanente no es una opción para un Técnico.

Vamos a configurar nuestro proyecto para que, cada vez que guarden su trabajo (git push), se suba automáticamente a dos lugares a la vez: GitHub (nuestro servidor principal) y GitLab (nuestro servidor de emergencia/backup). A esto se le llama **Dual-Remote**.

### PASO 1: Preparar la "Caja Fuerte" de Emergencia en GitLab

1. Entren a gitlab.com y abran su cuenta (pueden iniciar sesión directamente vinculando su cuenta de GitHub).
2. Hagan clic en "New Project" -> "Create blank project".
3. Pónganle exactamente el mismo nombre que usan en GitHub (ejemplo de repositorio: repasogeneral).
4. ⚠️ IMPORTANTE: NO marquen la casilla que dice "Initialize repository with a README". El proyecto tiene que crearse vacío.

### PASO 2: Configurar VS Code para que suba a los dos lados

Abran la terminal de PowerShell en VS Code (Ctrl + Ñ) dentro de la carpeta de su proyecto (repasogeneral) y ejecuten estos tres comandos tal cual:

1. Avisarle a Git que GitHub es nuestro destino principal

git remote set-url origin https://github.com/tu-usuario/repasogeneral.git

2. Agregar a GitHub como el PRIMER lugar donde guardar el código

git remote set-url --add --push origin https://github.com/tu-usuario/repasogeneral.git

3. Agregar a GitLab como el SEGUNDO lugar (el de emergencia)

git remote set-url --add --push origin https://gitlab.com/tu-usuario/repasogeneral.git

(No se olviden de cambiar `tu-usuario` por su nombre de usuario real en ambas páginas).

**PASO 3: Verificar que funcionó**

Escriban el siguiente comando en la terminal de VSCode:

git remote -v

Si la configuración es correcta, en pantalla se mostrarán 3 líneas (1 fetch y 2 push): una apuntando a GitHub y otra a GitLab:

origin https://github.com/tuusuario/repasogeneral.git (**fetch**)

origin https://github.com/tuusuario/repasogeneral.git (**push**)

origin https://gitlab.com/tuusuario/repasogeneral.git (**push**)

**PASO 4: Asegurar la rama y subir el código a ambos servidores**

Asegúrense de nombrar su rama principal como **main** y realicen el primer envío vinculado. Ahora, cada vez que terminen de programar, simplemente escriban los comandos que siempre usan. Git se encarga de subirlo a los dos repositorios con un Enter!:

git branch -M main
git add .
git commit -m "feat: configuro la proteccion **Dual-Remote** y guardo mis cambios"
git push -u origin main

A partir de este momento, cada vez que terminen de programar solo escribirán git push y se guardará en los dos repositorios al mismo tiempo.

💡 AUTENTICACIÓN EN GITLAB: La primera vez que suban a GitLab, Windows les abrirá una ventanita emergente pidiendo credenciales.

Opción rápida (Recomendada): Elijan iniciar sesión con el Navegador (Browser / OAuth). Se autoriza con un solo clic.

Plan B (Si no abre el navegador): Usen su usuario de GitLab y peguen un Personal Access Token (PAT) como contraseña. Este token se genera desde GitLab en Preferences > Personal Access Tokens (ejemplo: glpat-xxxxxxxxxxx).

## 📋 CUESTIONARIO TÉCNICO DE CONTROL

Investiguen y respondan estas 5 preguntas en su informe APA v7 en PDF para Moodle:

1. **Caso Base:** ¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?
2. **Eficiencia:** ¿Por qué la instrucción break optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?
3. **Direccionamiento:** ¿Cómo comprobar mediante el operador & que los elementos de un array ocupan celdas contiguas en la RAM?
4. **Variables Temporales:** ¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?
5. **Autoevaluación:** ¿Cuál de los 3 retos te resultó más complejo de comprender a bajo nivel y de qué manera lograste resolverlo?

## 🏆 PAUTA DE ENTREGA Y EVALUACIÓN

### ⚠️ Modalidad y Entorno Híbrido:

- **Formato de Entrega del informe APA v7 en PDF:** Puede ser grupal (P.I.A) o individual. Si es grupal, el informe APA debe incluir detallado el nombre de cada integrante y qué tarea exacta hizo cada uno. Archivo guardado en su carpeta local como Docs/InformeEEST1_LPR2026_ACT06_G03_Informe_v1.0.0.pdf
- Respuestas técnicas justificadas al Cuestionario de Control de 5 preguntas.
- Tabla de Control de Revisiones (DocVer) con versión `v1.0.0 Baseline Aprobado`.
- **Compilación (Local vs. Online):** El uso de VS Code y MSYS2 en la PC es para quienes tengan notebook/PC propia con permisos de administrador (por una limitación de permisos en algunos equipos de los laboratorios). Si las PCs de la escuela les bloquea o no les dejan compilar, es obligatorio usar OnlineGDB para correr el código (hagan las capturas correspondientes como evidencia).
- **El repositorio:** Sin importar si compilaste en tu computadora o en OnlineGDB, el repositorio propio de GitHub con toda la estructura de carpetas (src/, docs/, etc.) y el código subido con su modificación debe estar creado obligatoriamente. No se aceptan carpetas sueltas.
  Figura 1: Captura de la terminal ejecutando `git remote -v` mostrando claramente que Git está configurado para subir (push) a GitHub y a GitLab.
- **Carpeta de Campo analógica y digital del P.I.A. y Planilla de Proyectos:** Registro obligatorio de todas las actividades relacionadas exclusivamente con el P.I.A., no se deben dejar campos vacíos, se deben justificar con textos coherentes apuntando al P.I.A. (los campos son: **Objetivos para la semana; Entregas Parciales (APA + Código + Capturas); Obstáculos Encontrados y Acciones de Próxima Semana**).

## 🔗 Enlaces Técnicos de Referencia (Links de Consulta):

- 🔗Recursion in C++ - Conceptos de Recursividad
- 🔗Documentación de Microsoft C++ - Arrays y Arreglos
- 🔗GeeksforGeeks — Algoritmo de Búsqueda Secuencial (Linear Search)
- 🔗Tutorial de C++ - Punteros y Dirección de Memoria
- 🔗 Git Documentation — Git Remote & Dual Push Configurations

## 📅 Fecha de Entrega / Prórroga: 01 de Octubre de 2026.